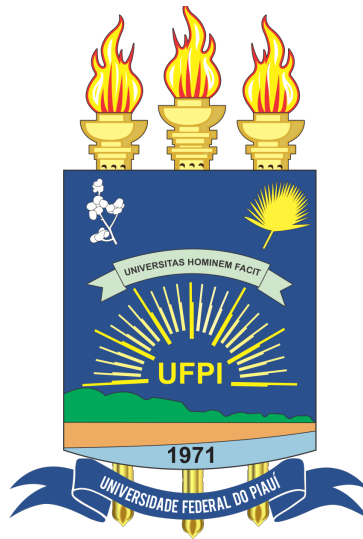


Documentação PDSI 1



Black Amber Coffe

Marcos Willian, Hermeosn Alves, Emanuely
mwss@outlook.com, hermeson.de@ufpi.edu.br

Contents

1	Contexto e Problema	2
2	Público-alvo	2
3	Escopo e Limites	3
3.1	Escopo Inicial	3
3.2	Escopo Fora da Versão	3
3.3	Critérios Para MVP	3
4	Metodologia de Desenvolvimento	3
4.1	Metodologia Scrum	3
4.2	Organização	3
5	Riscos e Restrições	4
5.1	Riscos	4
5.2	Restrições	4

6	Cronograma	4
6.1	Estrutura Geral	5
6.2	Planejamento por Sprints	5
7	Custos Envolvidos	6
7.1	Custos de Desenvolvimento	6
7.2	Custos de Infraestrutura	6
7.3	Ferramentas Utilizadas	6
7.4	Considerações Gerais	6
8	Tecnologias escolhidas	6
8.1	Front-end	7
8.2	Back-end	7
8.3	Observações Gerais	7

Abstract

Black Amber Coffee é uma cafeteria projetada para oferecer um ambiente ideal para trabalhadores, especialmente profissionais remotos e programadores. O espaço prioriza conforto, conectividade e condições adequadas para produtividade. Além disso, a cafeteria disponibiliza um sistema digital acessível via web e dispositivos móveis, permitindo que clientes realizem pedidos, efetuem pagamentos e acompanhem seu histórico de consumo de forma simples e eficiente. A solução também inclui um ambiente administrativo voltado aos colaboradores, responsável pela gestão das operações internas.

1 Contexto e Problema

Com o crescimento do trabalho remoto e da demanda por ambientes que favoreçam a produtividade fora do espaço doméstico, cafeterias passaram a ser utilizadas como locais alternativos de trabalho podemos até mesmo usar de exemplo a Starbucks. No entanto, muitos desses estabelecimentos não oferecem infraestrutura adequada, organização de pedidos eficiente ou ferramentas digitais que melhorem a experiência do usuário.

Nesse contexto, a Black Amber Coffee surge com a proposta de integrar um ambiente físico confortável a uma solução digital completa. O sistema permite que clientes realizem pedidos, pagamentos e acompanhem seu histórico de forma prática, enquanto os colaboradores contam com uma plataforma administrativa para gerenciar as operações da cafeteria. Dessa forma, busca-se otimizar tanto a experiência do cliente quanto a eficiência operacional do estabelecimento.

2 Público-alvo

O público-alvo da Black Amber Coffee é composto principalmente por profissionais remotos, freelancers e programadores que utilizam cafeterias como uma extensão de seus escritórios. Esse grupo terá grande interesse no local porque o espaço resolve diretamente as suas maiores necessidades diárias: oferece uma infraestrutura física otimizada para a produtividade, unindo conforto e conectividade de alta qualidade, aliada a uma experiência de consumo sem atritos.

A conveniência de um sistema digital atende clientes que valorizam uma experiência digital fluida, pois permite que eles façam pedidos, efetuem pagamentos e gerenciem seus gastos diretamente pelo celular ou notebook, garantindo que mantenham o foco no trabalho sem interrupções desnecessárias.

3 Escopo e Limites

3.1 Escopo Inicial

1. Conseguir efetuar pedidos de forma remota
2. Conseguir efetuar pagamentos de forma remota
3. Conseguir armazenar e acessar historicos de pedidos e pagamentos
4. Notificar promoções, cupons, pagamentos bem sucedidos, pedidos em espera/fazendo ou a caminho alem de possiveis eventos na cafeteria (Fechada, reforma e etc).
5. Gerenciar pedidos, organiza-los em prioridade.
6. Gerenciar pagamentos de clientes e funcionarios, historico de finanças e controle de perdas/ganhos.
7. Gerenciar funcionarios, entrada/saida, faltas, dias devendo e etc
8. Gerenciar contas de clientes e funcionarios.
9. Implemetação web
10. Implementação mobile

3.2 Escopo Fora da Versão

1. Possivel distribuição para a parte do cliente literalmente rodar em um APP (Aplicativo) dedicado.

3.3 Criterios Para MVP

1. Sistemas com 70% das funcionalidades propostas em funcionamento.
2. Ter o minimo de segurança.

4 Metodologia de Desenvolvimento

4.1 Metodologia Scrum

O Scrum foi escolhido por ser uma metodologia ágil, utilizada no gerenciamento e desenvolvimento de projetos, baseada em ciclos iterativos e incrementais chamados de Sprints, com o objetivo de promover entregas contínuas, adaptação a mudanças e colaboração entre a equipe.

4.2 Organização

- **Product Owner:** Responsável por definir e priorizar os requisitos do sistema, garantindo que as funcionalidades atendam às necessidades dos usuários.
- **Scrum Master:** Responsável por garantir que a equipe siga as práticas do Scrum, organizar as reuniões e remover impedimentos durante o desenvolvimento.
- **Time de Desenvolvimento:** Responsável pela implementação do sistema.

Neste projeto, as responsabilidades foram distribuídas da seguinte forma:

- **Emanuelly:** Responsável pela modelagem do sistema e prototipação, incluindo a elaboração de diagramas e definição estrutural da aplicação.
- **Hermeson:** Responsável pelo desenvolvimento do **back-end** e pela modelagem e gerenciamento do **banco de dados**.
- **Marcos Willian:** Responsável pelo desenvolvimento do **front-end**, incluindo a construção da interface e utilização do **Figma** para definição do design.

5 Riscos e Restrições

5.1 Riscos

- **Atrasos no desenvolvimento:** Devido à divisão de tarefas e dependências entre front-end e back-end, podem ocorrer atrasos na entrega das funcionalidades.
- **Problemas de integração:** Dificuldades na comunicação de front-end e back-end podem gerar erros no funcionamento.
- **Falta de experiência com metodologias:** A equipe pode enfrentar dificuldades no uso das ferramentas.
- **Mudanças de requisitos:** Alterações nas funcionalidades ao longo do desenvolvimento podem impactar o cronograma.
- **Problemas de segurança:** Falhas na autenticação ou proteção de dados podem comprometer o sistema.
- **Indisponibilidade de integrantes:** Ausência de membros da equipe pode afetar o andamento das Sprints.

5.2 Restrições

- **Tempo limitado:** O projeto deve ser concluído dentro do período da disciplina.
- **Equipe reduzida:** O projeto é desenvolvido por poucos integrantes, o que limita a quantidade de funcionalidades implementadas.
- **Dependência de internet:** O projeto depende de conexão com a internet para funcionamento.
- **Limitações de infraestrutura:** O projeto não contará com servidores robustos robustos, ou ambiente completo de produção.
- **Dependência entre módulos:** O front-end e o back-end depende da disponibilidade e funcionamento do back-end.
-

6 Cronograma

O desenvolvimento do sistema será realizado de forma incremental utilizando a metodologia Scrum, dividido em sprints flexíveis de 1 a 2 semanas, permitindo adaptação conforme a evolução do projeto.

6.1 Estrutura Geral

- Duração das Sprints: 1 a 2 semanas
- Entregas incrementais ao final de cada sprint
- Revisão e adaptação contínua do backlog

6.2 Planejamento por Sprints

1. Sprint 0 - Planejamento e Setup

- Definição da arquitetura do sistema
- Configuração do ambiente (Front e Back)
- Setup inicial do banco de dados (Drizzle + PostgreSQL/SQLite)
- Estrutura base do projeto (monorepo ou separado)

2. Sprint 1 - Autenticação e Base do Sistema

- Implementação de autenticação (JWT + Bcrypt)
- Criação de usuários (clientes e colaboradores)
- Estrutura inicial de rotas no backend
- Setup inicial de estado global (Zustand)

3. Sprint 2 - Pedidos

- Criação e listagem de pedidos
- Integração front-end com API (Axios)
- Validação de dados com Zod
- Interface básica para pedidos

4. Sprint 3 - Pagamentos

- Implementação do fluxo de pagamentos
- Registro de histórico de transações
- Feedback de status (sucesso, erro, pendente)

5. Sprint 4 - Notificações e Status

- Atualização de status de pedidos
- Notificações (pedido pronto, em preparo, etc)
- Estrutura para eventos da cafeteria

6. Sprint 5 - Painel Administrativo

- Interface para gerenciamento de pedidos
- Organização por prioridade/status
- Gestão básica de usuários

7. Sprint 6 - Refinamento e MVP

- Ajustes de usabilidade (UX/UI)
- Correção de bugs

- Validação do MVP (70% das funcionalidades)

8. Sprint 7+ - Evolução Contínua

- Melhorias de performance
- Novas funcionalidades
- Preparação para produção

7 Custos Envolvidos

Os custos do projeto Black Amber Coffee estão relacionados principalmente ao tempo de desenvolvimento da equipe e à utilização de ferramentas e infraestrutura tecnológica.

7.1 Custos de Desenvolvimento

- **Mão de obra:** O desenvolvimento é realizado pelos integrantes da equipe, não havendo custo financeiro direto, apenas investimento de tempo.
- **Tempo de desenvolvimento:** O projeto é desenvolvido ao longo da disciplina, distribuído em Sprints.

7.2 Custos de Infraestrutura

- **Hospedagem:** Pode haver custo futuro para hospedagem do sistema em servidores.
- **Banco de dados:** Utilização de PostgreSQL em produção, podendo gerar custos dependendo do serviço utilizado.
- **Internet:** Necessária para desenvolvimento, testes e uso do sistema.

7.3 Ferramentas Utilizadas

- **Figma:** Utilizado para prototipação (versão gratuita).
- **Visual Studio Code:** Ambiente de desenvolvimento gratuito.
- **GitHub:** Controle de versão gratuito.
- **Node.js e bibliotecas:** Ferramentas open-source sem custo.

7.4 Considerações Gerais

Grande parte das ferramentas utilizadas no projeto são gratuitas ou possuem planos free, reduzindo significativamente os custos financeiros diretos. O principal investimento está relacionado ao tempo e esforço da equipe.

8 Tecnologias escolhidas

A escolha das tecnologias foi baseada na necessidade de produtividade, escalabilidade e padronização do desenvolvimento, priorizando o uso de TypeScript para garantir tipagem estática e redução de erros.

8.1 Front-end

- **Figma:** Ferramenta utilizada para prototipação e definição da interface do sistema.
- **React + Vite:** Base da aplicação front-end, proporcionando desenvolvimento rápido e modular.
- **TypeScript:** Linguagem principal, garantindo tipagem estática e maior segurança no código.
- **Tailwind CSS:** Framework utilitário para estilização rápida e consistente.
- **ShadCN UI:** Biblioteca de componentes reutilizáveis para padronização da interface.
- **Axios:** Cliente HTTP para comunicação com a API.
- **Zustand:** Gerenciamento de estado leve, utilizado para cache e controle global.
- **Zod:** Biblioteca para validação de dados no front-end.

8.2 Back-end

- **Node.js + Express:** Ambiente e framework para construção da API REST.
- **TypeScript:** Utilizado para padronização e segurança do código no servidor.
- **Drizzle ORM:** Ferramenta para manipulação e abstração do banco de dados.
- **Zod:** Validação de dados no back-end.
- **Bcrypt:** Utilizado para hash de senhas e segurança na autenticação.
- **JWT:** Implementação de autenticação baseada em tokens.
- **UUID:** Geração de identificadores únicos para entidades do sistema.
- **Dotenv:** Gerenciamento de variáveis de ambiente.
- **Swagger:** Documentação interativa da API.
- **PostgreSQL:** Banco de dados no ambiente de produção.

8.3 Observações Gerais

- O projeto utiliza TypeScript em toda a stack para reduzir erros e melhorar a manutenibilidade.
- A arquitetura prioriza modularização e separação de responsabilidades.
- Algumas estruturas foram planejadas de forma genérica visando possível expansão futura.